

FREE RESOURCE

React Debugging Cheatsheet

A practical workflow for developers who want to find bugs faster, understand what changed, and stop guessing.

What you will get
☐ A repeatable debugging workflow you can use on real projects
☐ The most common React failure points to check first
☐ A cleaner way to isolate state, props, effects, and render issues

Built for quick wins first. Use this checklist to spot obvious problems, prioritize fixes, and create momentum before you spend money on a full rebuild.

How to use this PDF
☐ Review each page once from top to bottom
☐ Mark the items that are already done
☐ Circle the 3 to 5 highest-impact fixes
☐ Turn those fixes into your next sprint or support request

Free resource by Zaid Ahmad

Shopify and WordPress developer. 1:1 technical support, speed optimization, implementation, and tutoring.

Debugging mindset first

Good debugging is not random trial and error. Start by shrinking the problem until you can explain it in one sentence.

Before touching code, write down:

- ☐ What exactly is broken?
- ☐ Where does it happen?
- ☐ What should happen instead?
- ☐ What changed right before the bug appeared?
- ☐ Can I reproduce it every time?

Fast isolation checks

- ☐ Reproduce with the smallest possible steps
- ☐ Test with fake data if live data is noisy
- ☐ Remove unrelated UI until the issue is isolated
- ☐ Confirm whether the bug is visual, state-related, or data-related

Questions that save time

- ☐ Is the component rerendering when I expect?
- ☐ Is the prop value actually what I think it is?
- ☐ Did the state update happen before this code ran?
- ☐ Is there an effect causing unexpected behavior?

Common React bug categories

Check these first

- ☐ Incorrect prop names or missing props
- ☐ State updates based on stale values
- ☐ useEffect dependency mistakes
- ☐ Conditional rendering hiding the wrong UI
- ☐ Keys missing or unstable in lists
- ☐ Async data arriving later than the component expects
- ☐ Controlled vs uncontrolled input issues

A simple debugging workflow

Use the same flow every time. Consistency matters more than cleverness when the codebase gets messy.

The workflow
1. Reproduce the bug reliably
2. Add logs at the edges: props in, state changes, event handlers out
3. Confirm what renders and when
4. Remove assumptions and inspect actual values
5. Change one thing at a time
6. Retest after each change

What to log
☐ Incoming props
☐ Current state before and after updates
☐ Effect triggers and dependency values
☐ API responses and transformed data

What not to do
☐ Do not change five files at once
☐ Do not trust memory over actual logs
☐ Do not skip the reproduction steps
☐ Do not keep dead debugging code after the fix

useEffect checks

If <code>useEffect</code> is involved, verify:
☐ Every dependency is intentional
☐ The effect is not updating state in a loop
☐ Async logic handles loading and error states cleanly
☐ Cleanup runs when subscriptions or timers are used
☐ The effect is not doing work that belongs in an event handler

State, props, and rendering

When the UI looks wrong, first figure out whether the data is wrong or the rendering logic is wrong. That split makes debugging much easier.

If data is wrong	If UI logic is wrong
☐ Check fetch timing and default state	☐ Review conditions for showing and hiding elements
☐ Validate object shape before rendering	☐ Check class names and state-driven variants
☐ Confirm derived values are recalculated correctly	☐ Confirm list keys and ordering
☐ Watch for stale closures in handlers or effects	☐ Test empty, loading, and error states

When you are stuck for more than 20 minutes
☐ Rebuild the failing case in a smaller component
☐ Comment out half the logic to narrow the source
☐ Pair with another developer and explain the bug out loud
☐ Step away for 10 minutes and come back with a fresh hypothesis

Final pre-ship checklist

Before you call it fixed
☐ The original bug is gone
☐ The fix works on the main edge cases too
☐ No warnings were introduced
☐ You removed temporary debugging code
☐ You understand why the bug happened

Want hands-on help with React?

I offer 1:1 debugging support, project help, and practical tutoring for React, JavaScript, Node.js, Shopify, and WordPress. Book a session at zaidahmaddev.com.